État d'avancement — Agent IA de revue des PR

Nov 5, 2026 · @oussama

Synthèse

Le lot des semaines 1-2 (prototype) est terminé et testé de bout en bout : l'agent reçoit une PR, l'analyse avec un modèle local et publie commentaires, statut et alerte. Il tourne en local sous Docker, sans être encore branché sur un vrai dépôt.

Le point faible est la qualité du modèle : sur le banc de test, environ 2 remarques sur 5 sont pertinentes, contre 70 % visés. Pour avancer vers le pilote, il faut un serveur GPU, la création de la GitHub App dans l'organisation et le choix d'un dépôt pilote.

Réalisé (semaines 1-2)

Le prototype couvre toute la chaîne prévue au cahier des charges, sauf les contrôles statiques et le chargement du contexte (lots suivants). Stack : LangGraph + FastAPI, Redis, Docker, Ollama (qwen2.5-coder).

Ce que fait l'agent à chaque PR ouverte ou modifiée :

1. Reçoit le webhook GitHub, vérifie sa signature et le met en file (doublons ignorés).
2. Lit la PR ; si un commit plus récent existe, l'analyse obsolète s'arrête.
3. Filtre le diff : lock files, fichiers générés, `vendor/`, binaires exclus ; limite de taille par fichier et par PR.
4. Détecte les secrets et les masque avant tout envoi au modèle.
5. Fait relire le diff par le modèle local, découpé en lots, avec les règles de l'équipe.
6. Écarte les remarques inventées, en double ou trop mineures, puis calcule le verdict : bloquante, à vérifier ou prête.
7. Publie une revue avec commentaires en ligne et un statut sur la PR.
8. Envoie un email si la PR est bloquante et journalise l'analyse.

Garde-fous sécurité livrés :

- **Aucun code ne sort** : l'agent refuse d'appeler un modèle dont l'hôte n'est pas autorisé ; le traçage cloud de LangGraph (LangSmith) est coupé.
- **Droits minimaux** : le bot lit le code, écrit commentaires et statuts, ne peut ni fusionner ni modifier le code, et n'approuve jamais.
- **Secrets** : jamais recopiés dans un commentaire, un email ou le journal.

- **Journal d'audit** : une ligne par analyse, sans code (verdict, compteurs, modèle, durée).

Livrables : dépôt `C:\Projects\agentGTIHUB`, 2 commits sur `main` (`cb282f5` prototype, `388dac0` banc de test). Dépôt local uniquement, pas encore poussé sur GitHub.

Tests et résultats

Tout ce qui ne dépend pas du modèle fonctionne : 33 tests unitaires passent et le banc de bout en bout passe sur ses 4 PR de test. Le banc utilise le vrai agent, Redis et Ollama, avec un faux GitHub et une fausse boîte mail : rien n'est envoyé à l'extérieur.

PR de test	Résultat vérifié	Temps (7b, CPU)
Contrôleur avec clé AWS en dur, injection SQL, `dump()`	Statut « bloquante », email envoyé, commentaire sur la ligne du secret ; le secret n'apparaît nulle part	85 s (215 s au 1er passage, chargement du modèle)
Service propre + son test	Revue publiée, verdict « prête »	53 s
Fichier de 2 000 lignes	Signalé « trop volumineux », verdict « à vérifier »	40 s
Webhook sur un ancien commit	Ignoré, rien publié	< 1 s
Même webhook reçu deux fois	Second envoi refusé	< 1 s

Qualité des remarques du modèle sur le contrôleur vulnérable :

Problème réel	qwen2.5-coder 0.5b	qwen2.5-coder 7b
Injection SQL	Non trouvée	Trouvée, classée « majeur » au lieu de « bloquant »
`dump()` oublié	Non trouvé	Trouvé
Route de suppression sans contrôle d'accès	Non trouvée (remarque sur la bonne ligne, mais vide de sens)	Non trouvée
Fausses alertes	Une remarque « bloquante » inventée sur un fichier propre	2 remarques mineures inutiles, sans effet sur le verdict

Le 0.5b est inutilisable pour la revue ; le 7b est la base de travail. L'agent a aussi été lancé en lecture seule sur 2 PR publiques de `symfony/demo` (24 s et 10 s avec le 0.5b).

Trois défauts trouvés par les tests ont été corrigés : le worker plantait sur une attente Redis, le modèle recommentait le secret masqué (doublon), et la sortie console Windows plantait sur un caractère.

Écarts et risques

Un critère de validation est déjà tenu sur le banc (aucune fuite) ; la pertinence est en dessous de l'objectif ; les deux autres ne se mesureront qu'au pilote.

Critère de validation	Mesuré à ce jour	État
100 % des PR analysées en moins de 10 min	40 s à 3 min 35 par petite PR, sur CPU ; pas encore testé sur des PR réelles volumineuses	À confirmer avec GPU
≥ 70 % des anomalies jugées pertinentes	Environ 40 % (2 sur 5) avec le 7b, sur un échantillon très petit	Non atteint
Aucune fuite de code	Modèle local seul autorisé ; secret introuvable dans GitHub, emails, journal et logs	Tenu sur le banc
Baisse du temps de revue humaine	Pas mesurable avant le pilote	À mesurer

Risques constatés :

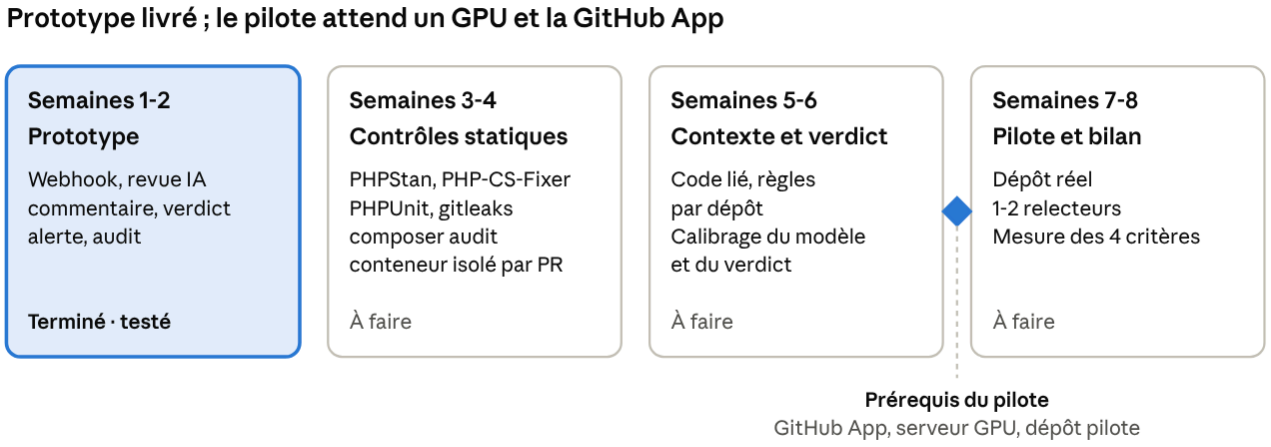
- **Qualité du modèle local** (risque prévu au cahier des charges, confirmé) : le 7b rate un contrôle d'accès et sous-classe une injection SQL. Pistes : modèle plus gros sur GPU, travail sur le prompt et les règles, ou cloud si la conformité l'accepte.
- **Faux positifs** : aujourd'hui limités à des remarques mineures, mais un petit modèle a déjà inventé une remarque bloquante. Le verdict bloquant repose donc pour l'instant surtout sur le détecteur de secrets, et demain sur les contrôles statiques.
- **Sans GPU**, les PR de taille réelle risquent de dépasser 10 minutes.

Limites connues du prototype, à traiter avant la production :

- une nouvelle revue est publiée à chaque commit, sans nettoyer les remarques précédentes ;
- si le worker s'arrête pendant une analyse, celle-ci est perdue ;
- l'alerte part vers une liste fixe, pas vers l'auteur (GitHub ne fournit pas son email).

Prochaines étapes et besoins

Le lot suivant est celui des contrôles statiques (semaines 3-4) ; il peut démarrer tout de suite, sans attendre les prérequis du pilote.



planning · 4 lots de 2 semaines, 1 prérequis avant le pilote

Les contrôles statiques rendront le verdict bloquant moins dépendant du modèle ; le pilote ne peut commencer qu'une fois les trois prérequis réunis.

Besoins pour continuer :

- [ ] Serveur GPU pour le modèle local (appui DevOps)
- [ ] Création de la GitHub App dans l'organisation et installation sur le dépôt pilote (administrateur GitHub)
- [ ] Choix du dépôt pilote
- [ ] Relais SMTP interne et liste des destinataires des alertes

- [ ] Dépôt distant pour le code de l'agent (aujourd'hui local uniquement)
- [ ] 1 à 2 relecteurs pour juger la pertinence des remarques pendant le pilote
- [ ] Décision conformité : un modèle cloud est-il envisageable si le local reste insuffisant ?